Ans1) Time Complexity is the amount of time an algorithm takes to run as the input size (n) increases.

Ans 2) Space Complexity is the amount of memory (RAM) an algorithm uses as the input size (n) increases.
It includes:
- Input Space (memory for the input)
Auxiliary Space (extra memory used by the algorithm)

Ans 3) Asymptotic notations describes how an algorithm's time or space complexity grows as the input size (n) becomes very large.

(1) Big O (O) - Worst Case
- Represents the maximum time an algorithm can take.
- Gives the upper bound of complexity.

(2) Omega (Ω) - Best case
- Represents the minimum time an algorithm can take.
- Gives the lower bound of complexity

(B) Theta (Θ) - Average / Exact Bound
- Represents the tight (exact) bound when the algorithm grows at the same rate in both upper and lower bounds.
- Used when the running time is consistently proportional.

eg: Traversing an array always require visiting all element → $O(n)$

Ans 4)

I) BEST CASE
- Minimum time an algorithm takes.
- Happens in the most favourable situation.

eg: Time complexity: $O(1)$

II) AVERAGE CASE
- Expected time for a typical input
- Represents the algorithm's performance on average.

eg: Time complexity: $O(n)$

III) WORST CASE
- Maximum time an algorithm takes
- Happen in the least favourable situation

eg: Time ~~tot~~ complexity: $O(n)$

Ans5)

Code Patterns	Time complexity
Single Statement	O(1)
One loop	O(n)
Two nested loops	O(n²)
Consecutive loops	add, then simplify
Ignore constants	O(1) + O(n) + O(n²)
Keep highest term	eg O(n² + n) = O(n²)
Halving / Doubling loop	O(log n)

Ans6)

loop Pattern	Time complexity
One loop	O(n)
Two nested loop	O(n²)
consecutive loop	O(n)

Ans7)

Nested loop type	Time complexity
2 nested loop (n x n)	O(n²)
Inner loop j < i	O(n²)
Outer n, Inner m	O(n*m)

Ans8)

loop Pattern	Time Complexity
i *= 2	O(log n)
i /= 2	O(log n)
Search space halves each step	O(log n)

Ans 9)

Memory Used	Space Complexity
few variables only	O(1)
one extra array of size(n)	O(n)
n recursive calls	O(n)

MODULE 1

Time Complexity: All the patterns shown are $O(n^2)$ because they are nested loops and the total number of printed character grows proportionally to n^2.

Space Complexity: All have O(1) auxiliary space because they use only a few loop variables and do not allocate extra memory based on n

MODULE 2

Time and Space complexity of Searching and Sorting Algorithms

Best Case	Average case	Worst case	Space Complexity	Brief Explanation
$O(1)$	$O(n)$	$O(n)$	$O(1)$	checks elements one by one. Best case if the element is first worst if it is last or absent.
$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ $O(\log n)$	Repeatedly halves the search space. Works only on sorted arrays.
$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Repeatedly swaps adjacent elements. Largest element moves to the end after each pass.

MODULE 3

Best Case	Avg Case	Worst Case	Space Complexity	Space Complexity
$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	divides the array into halves sorts each half then merges them performance is always $O(n \log n)$

$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	chooses a pivot and partitions the array around it. Very fast in practise, but worst when the pivot is always the smallest or largest element.

MODULE 4

A1) Time Complexity O(n)
The for loop runs from 1 to N, printing one number in each iteration. Since there are N iterations the running time grows linearly with N.
Space complexity.
O(1)
The program uses only a few variables (N and i) regardless of the input size.

A2) Time Complexity O(n)
Since the loop executes N times.
Space Complexity.
O(1)
The program uses only a fixed number of variables (N, SUM and i). No extra memory.

Ans 3)

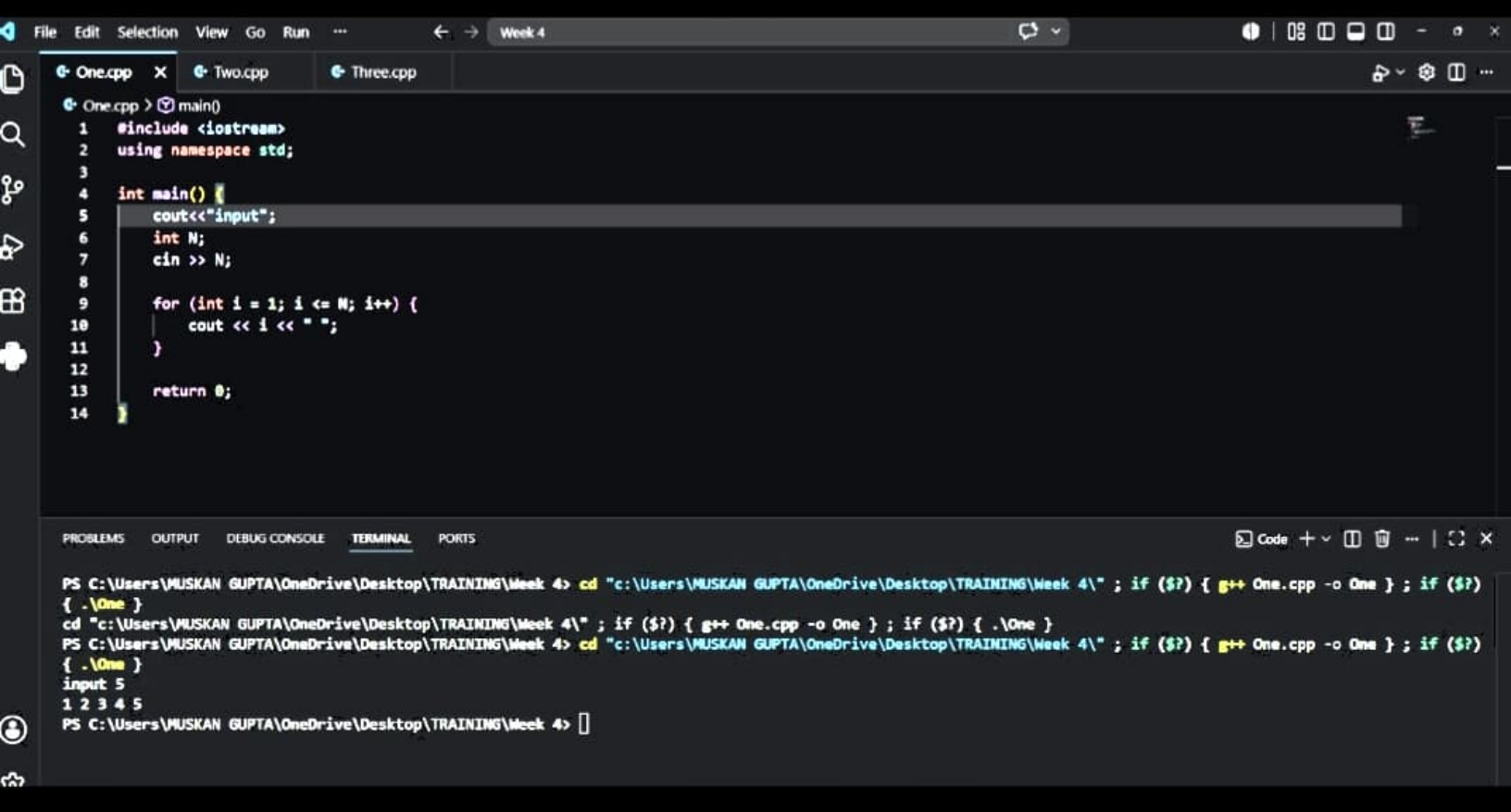
O(B) Time Complexity

The loop runs B times and in each iteration one multiplication is performed.

Space Complexity

O(1)

The program uses only a few var


File Edit Selection View Go Run ···
Week 4
One.cpp
Two.cpp
Three.cpp
One.cpp > main()
#include <iostream>
using namespace std;
int main() {
cout<<"input";
int N;
cin >> N;
for (int i = 1; i <= N; i++) {
cout << i << " ";
}
return 0;
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code
PS C:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4> cd "c:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4\" ; if ($?) { g++ One.cpp -o One } ; if ($?) { .\One }
cd "c:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4\" ; if ($?) { g++ One.cpp -o One } ; if ($?) { .\One }
PS C:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4> cd "c:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4\" ; if ($?) { g++ One.cpp -o One } ; if ($?) { .\One }
input 5
1 2 3 4 5
PS C:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4>

File Edit Selection View Go Run ··· Week 4

One.cpp | Two.cpp | Three.cpp

Two.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main() {
    int N;
    cout<<"input";
    cin >> N;

    int sum = 0;

    for (int i = 1; i <= N; i++) {
        sum += i;
    }

    cout << sum << endl;

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Code

```
PS C:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4> cd "c:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4\" ; if ($?) { g++ Two.cpp -o Two } ; if
{ .\Two }
input5
15
PS C:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4>
```

File Edit Selection View Go Run ···

Week 4

One.cpp | Two.cpp | Three.cpp

Three.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main() {
    int A, B;
    cout<<"enter : ";
    cin >> A >> B;

    int result = 1;

    for (int i = 1; i <= B; i++) {
        result = result * A;
    }

    cout << result << endl;

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Code

```
enter : cd "c:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4\" ; if ($?) { g++ Three.cpp -o Three } ; if ($?) { .\Three }
0
PS C:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4> cd "c:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4\" ; if ($?) { g++ Three.cpp -o Three } ; if
$?) { .\Three }
enter : 2 5
32
PS C:\Users\MUSKAN GUPTA\OneDrive\Desktop\TRAINING\Week 4>
```